# tmux documentation

# tmx-state — Operator Guide

**Revision:** 1 **Last modified:** 2026-05-22T14:30:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** Operator inspecting / overriding / resetting the per-session cwd state file used by the v1.0.9 `tmx-shell-init.sh` and `tmx-ssh-dispatch.sh` integrations

---

## 1. Overview

`tmx-state` is a small Go binary (single static executable, no runtime dependencies) that stores one record per tmx session in `~/.tmx/state.json`. The wrapper `scripts/tmx` queries it on `tmx new` to restore the session's last cwd.

**Recording mechanism (v1.0.13+):** the cwd is recorded by a `PROMPT_COMMAND` (bash) / `precmd_functions` (zsh) hook installed by `tmx-shell-init.sh` inside every tmux pane. Every shell prompt fires `tmx-state record <session> $PWD`. The state file therefore reflects the cwd at the LAST prompt before `exit`. v1.0.9–v1.0.12 relied on tmux's `client-detached` + `session-closed` hooks alone; those hooks fire AFTER the pane is destroyed and `#{pane_current_path}` resolves to empty in that context. v1.0.13 keeps the session-end hooks as best-effort fallback but the prompt-hook is the primary mechanism.

Why a Go binary instead of a shell helper:

- **Atomic writes.** A temp-file write + `rename(2)` is atomic on POSIX; shell-only equivalents are racy under concurrent panes.
- `fcntl(F_SETLKW)` **locking.** Ten panes hitting `record` at once serialize cleanly; latest-wins.
- **Sub-millisecond reads.** No interpreter startup, no JSON parsing in shell.

The binary is built by `bash scripts/setup.sh` and lives at `scripts/tmx-state-bin`. The same source tree generates a `tmx-state` symlink in the project `scripts/` directory so plain `tmx-state` works once `scripts/` is on `$PATH`.

Design authority: `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md` §4.B + §5.1 + §5.3.

## 2. CLI reference

```
tmx-state record  <session> <abs-path>
tmx-state recall  <session>
tmx-state list
tmx-state forget  <session>
tmx-state version
tmx-state help
```

All subcommands respect `$TMX_STATE_FILE` for the file path (default `~/.tmx/state.json`).

## 2.1 `record <session> <abs-path>`

Atomically upserts a session's `last_pwd` and bumps `last_seen_unix`. Creates `~/.tmx/` mode 0700 + `state.json` mode 0600 if missing.

```
$ tmx-state record work /tmp
$ echo $?
0
```

Errors:

- exit 1 + stderr if `<session>` is empty or `<abs-path>` is relative.

## 2.2 `recall <session>`

Prints `last_pwd` to stdout (NO trailing newline — safe for `$(tmx-state recall …)` command substitution). Exit codes:

| Code | Meaning |
|---|---|
| 0 | Found — last_pwd printed |
| 1 | Not found (session never recorded, or corruption rebuilt empty) |
| 2 | State file unreadable / corrupt (treat as "unknown", fall back) |

```
$ tmx-state recall work
/tmp
$ tmx-state recall never-existed
$ echo $?
1
```

The wrapper uses this exit code to decide between `-c $(tmx-state recall X)` and falling back to `$HOME`.

## 2.3 `list`

Tab-separated table sorted by session name:

```
SESSION<TAB>LAST_PWD<TAB>LAST_SEEN_UNIX
```

```
$ tmx-state list
build   /Users/milosvasic/Projects/tmux         1748189000
investigate     /tmp                             1748190100
work    /Users/milosvasic/Projects/tmux/docs    1748190500
```

## 2.4 `forget <session>`

Idempotent removal. Exit 0 whether the session was present or not.

```
$ tmx-state forget investigate
$ tmx-state recall investigate
$ echo $?
1
```

### 2.5 `version`

```
$ tmx-state version
tmx-state v1.0.9
```

### 2.6 `help` / `--help` / `-h`

Prints the same usage block to stdout (exit 0).

## 3. State file location and override

Default path: `$HOME/.tmx/state.json` mode 0600 (parent dir mode 0700).

Override via `$TMX_STATE_FILE`. Useful for:

- isolated test runs (tests 27, 33, 38, 39, 40 do this);
- per-project state files;
- sandboxes where `$HOME` is read-only.

```
$ TMX_STATE_FILE=/tmp/my-tmx-state.json tmx-state record work /
        tmp
$ TMX_STATE_FILE=/tmp/my-tmx-state.json tmx-state list
work    /tmp    1748190500
```

## 4. State file schema

```
{
  "schema_version": 1,
  "sessions": {
    "work": {
      "last_pwd": "/Users/milosvasic/Projects/tmux",
      "last_seen_unix": 1748000000,
      "created_unix": 1747000000,
      "host": "nezha.local"
    },
    "investigate": {
      "last_pwd": "/tmp",
      "last_seen_unix": 1748190100,
      "created_unix": 1748190100,
      "host": "mistborn.local"
    }
  }
}
```

Single-file design (not jsonl). Fits in memory trivially — kilobytes even for hundreds of sessions. Atomic write-then-rename. Easy to inspect by hand:

```
$ cat ~/.tmx/state.json | python3 -m json.tool
```

The `host` field is the value of `os.Hostname()` at record-time. It's informational — the binary does not gate on it (a session named `work` on macOS and `work` on nezha are independent files, since each host has its own `~/.tmx/state.json`).

# 5. Worked examples

### Example 1 — pretty-print the current state

```
$ cat ~/.tmx/state.json | python3 -m json.tool
{
    "schema_version": 1,
    "sessions": {
        "work": {
            "last_pwd": "/Users/milosvasic/Projects/tmux",
            "last_seen_unix": 1748190500,
            "created_unix": 1747000000,
            "host": "mistborn.local"
        }
    }
}
```

### Example 2 — clean slate (reset everything)

```
$ rm ~/.tmx/state.json
$ tmx-state list
$ echo $?
0
# state file is missing; tmx-state silently treats this as an
        empty store.
# The next `tmx new -s X` rebuilds the file on first record.
```

### Example 3 — query from a script

```
#!/usr/bin/env bash
SESSION="work"
PWD="$(tmx-state recall "$SESSION" 2>/dev/null)"
if [ -n "$PWD" ] && [ -d "$PWD" ]; then
    echo "would start session '$SESSION' at $PWD"
else
    echo "no cwd memory for '$SESSION' — would start at \$HOME"
fi
```

### Example 4 — concurrent record from 10 panes

```
# Inside an existing tmx session named 'storm' — open 10 panes,
# each cd's to /tmp/pane-N and immediately detaches.
$ for i in $(seq 1 10); do
    (mkdir -p "/tmp/pane-$i" && tmx-state record "storm-$i" "/
        tmp/pane-$i") &
  done
$ wait
$ tmx-state list | wc -l
10
# fcntl(F_SETLKW) serialized the 10 writers without dropping any
        record.
```

(This is exactly what test `33_state_concurrency.sh` asserts.)

### Example 5 — recover from a corrupt state file

```
$ echo 'not valid json' > ~/.tmx/state.json
$ tmx-state list
tmx-state: notice: state file was corrupt, rebuilt
$ tmx-state recall work
$ echo $?
2     # ← unreadable signal; wrapper falls back to $HOME on
        `new`.
```

## 6. Troubleshooting

| Symptom | Diagnosis | Fix |
|---|---|---|
| `tmx-state: command not found` | `scripts/` not on `$PATH` | `bash scripts/setup.sh` re-runs the snippet installer |
| `recall` always returns nothing | State file missing or empty | Detach from a session at least once — the tmux hook will write the record |
| Wrong cwd restored | Stale state — directory was moved or deleted | `tmx-state record SESSION /new/path` to overwrite, or `forget` + recreate |
| `tmx-state record: pwd must be absolute, got "foo"` | You passed a relative path | Pass an absolute path; use `$(pwd)` if you need the current directory |
| `~/.tmx` unwritable | Filesystem read-only or wrong owner | `chmod 700 ~/.tmx && chown $(id -u):$(id -g) ~/.tmx` |
| State file mode looks wrong | An external tool touched it | `chmod 600 ~/.tmx/state.json` — restores §11.4.10 default |

## 7. Cross-references

- docs/guides/tmx-shell-integration.md — the rc-side prompt that calls `recall` indirectly

- docs/guides/tmx-ssh-dispatch.md — SSH dispatcher that also calls `recall`
- docs/manual/tmx-shell-integration.md — end-user master manual
- docs/scripts/tmx-state.md — §11.4.18 script companion (schema, internals)
- Spec: `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md` §4.B + §5.1 + §5.3

# 8. Anti-bluff (§11.4)

Tests 27 / 33 / 38 / 39 cover:

- 27 `state_persistence.sh` — record → recall round-trip + real tmux hook firing on session-close (positive evidence: pane_current_path read back via `display-message`);
- 33 `state_concurrency.sh` — 10 parallel records, all 10 keys present, JSON still parses;
- 38 `stale_pwd_fallback.sh` — recorded path deleted → wrapper falls back to `$HOME` cleanly;
- 39 `state_unwritable.sh` — `chmod 000 ~/.tmx` → `tmx new` still works (no cwd restore, no crash).

Paired mutation M21 strips the cwd-capture hook from `scripts/tmx.template` → test 27 FAILs (the hook stops writing, so the second `recall` returns the OLD path, not the post-`cd` path). Restored → PASS.

# 9. Last verified

2026-05-22 on Darwin arm64 (macOS 15.x, Go 1.22, tmux 3.6a) with `bash scripts/tests/27_state_persistence.sh`, `33_state_concurrency.sh`, `38_stale_pwd_fallback.sh`, and `39_state_unwritable.sh` all PASS, 3/3 iterations identical evidence-hash per §11.4.50.